

Draft-Length Scaling of Speculative Decoding Methods in DeepSpec

Technical Task Report

Abstract

Speculative decoding reduces autoregressive inference cost by letting a lightweight draft model propose several tokens that are verified in parallel by a target model. This work compares three recent drafting approaches implemented in DeepSpec: EAGLE-3, DFlash, and DSpark. The target model is Qwen3-4B, and evaluation is performed on GSM8K, Math500, and AIME25. We introduce an inference-time draft-length control that sweeps $D = 1, \dots, 7$ using the released checkpoints trained for a maximum proposal length of seven. We measure accepted length and end-to-end generation latency, while also reporting normalized throughput. On the two 500-sample benchmarks, longer proposals benefit DFlash and especially DSpark substantially more than EAGLE-3. EAGLE-3 is competitive at short draft length, whereas DSpark provides the strongest joint accepted-length/latency trade-off for larger D . The results illustrate the practical trade-off between sequential draft quality, parallel draft cost, and intra-block dependency modeling.

1 Introduction

Autoregressive decoding is inherently sequential: every newly generated token depends on the prefix produced so far. This becomes particularly expensive for reinforcement-learning rollout generation, where many long responses are sampled from the same target model. Speculative decoding reduces this cost by separating generation into two stages: a smaller draft model proposes several future tokens, and the target model verifies them in parallel.

The main design question is therefore not only how accurate the draft model is, but how proposal quality changes as the draft length D grows. A longer draft can amortize target-model verification over more output tokens, but only if the draft tokens are accepted often enough and the draft model itself remains cheap.

We compare EAGLE-3 [4], DFlash [1], and DSpark [2] using the DeepSpec implementation [3]. The target model is Qwen3-4B [5]. DeepSpec publishes Qwen3-4B checkpoints for all three methods with maximum proposal length seven and evaluates speculative decoding on the same benchmark family used here, including GSM8K, Math500, and AIME25 [3].

Our study focuses on two quantities required by the task: (1) accepted length as a function of draft length, and (2) generation latency as a function of draft length. The key experimental contribution is a controlled inference-time sweep $D \in \{1, \dots, 7\}$ in which changing D changes the actual draft computation, rather than computing a length-seven proposal and slicing it afterward.

2 Methods

EAGLE-3. EAGLE-3 improves the EAGLE family by replacing feature prediction with direct token prediction and by fusing target-model features from multiple layers through a training-time-test mechanism [4]. In the DeepSpec evaluation path used here, draft tokens are proposed sequentially. Consequently, increasing D increases both the number of speculative tokens available for verification and the sequential draft work.

DFlash. DFlash targets the sequential drafting bottleneck directly. It uses a lightweight block-diffusion drafter that proposes a block of tokens in parallel in a single draft-model forward pass, while conditioning on features extracted from the target model [1]. This makes the draft cost scale differently with D than the autoregressive EAGLE-3 path.

DSpark. Purely parallel drafting can lose proposal quality for later positions because tokens inside the block do not explicitly depend on previously sampled draft tokens. DSpark therefore combines a parallel backbone with a lightweight sequential component, producing a semi-autoregressive drafter with explicit intra-block dependency

modeling [2]. DSpark additionally introduces confidence-scheduled verification for serving. In our controlled draft-length study, the confidence threshold is fixed to zero so that the requested D remains the independent variable; the semi-autoregressive/Markov mechanism and confidence head computation remain enabled.

3 Experimental Methodology

3.1 Setup

All experiments use Qwen3-4B in non-thinking mode as the target model. The released DeepSpec checkpoints are `eagle3_qwen3.4b_ttt7`, `dflash_qwen3.4b_block7`, and `dspark_qwen3.4b_block7`. The benchmark uses 500 GSM8K samples, 500 Math500 samples, and 30 AIME25 samples. Experiments were executed on NVIDIA RTX 3090 GPUs with 24 GB memory, using one benchmark process per GPU. The main software environment used Python 3.11, PyTorch 2.9.1+cu126, Transformers 5.10.2, and Triton 3.5.1.

Generation uses `max_new_tokens=2048`, temperature 1.0, seed 980406, and `confidence_threshold=0.0`. The DeepSpec source baseline is commit `005e03b81cec38b7da6399833d609ee89a2587f2`.

3.2 Runtime draft-length ablation

We added a runtime argument `--draft-length D`, constrained by

$$1 \leq D \leq D_{\max} = 7.$$

For EAGLE-3, the proposal loop performs exactly D sequential proposal steps. For DFlash and DSpark, the draft block allocated and forwarded through the model has length D . Thus the latency sweep changes real draft compute. Each speculative round verifies the current token together with the D proposed draft tokens. The target-model verification input therefore has length $D + 1$:

$$[x_t, d_1, \dots, d_D].$$

The first D target distributions verify the proposed draft tokens; the final distribution supplies the corrective token after rejection or the bonus token when all draft tokens are accepted.

3.3 Metrics and timing

The primary acceptance metric is DeepSpec’s mean *acceptance length*. In a normal non-EOS round, this is the accepted draft prefix plus one target-model corrective/bonus token. We additionally record `accepted_draft_tokens_mean`, which excludes that target token.

Latency is measured around the complete `generate_one_sample()` call. CUDA is synchronized immediately before and after the timed region so the CPU timer observes completed GPU work. Model loading, dataset loading, tokenization, and JSON serialization are outside this interval. We also report generated tokens/s and ms/generated-token to expose possible differences in stochastic output length.

To reduce correlation between D and slow run-level performance drift, configurations were not executed monotonically from 1 to 7. The main draft-length order was

$$4, 1, 7, 2, 6, 3, 5,$$

with method order also rotated between groups.

All aggregate result JSON files were cross-validated against their per-sample records: per-sample latency sums exactly matched aggregate latency, and per-sample output-token counts exactly matched aggregate generated-token counts.

4 Results

Figure 1 shows that accepted length increases with D , but not at the same rate for all methods. On GSM8K and Math500, DSpark develops an increasing advantage as the proposal grows. DFlash remains between DSpark and EAGLE-3 at larger D , while EAGLE-3’s accepted-length growth is weaker. This is consistent with DSpark’s explicit intra-block dependency modeling: later draft positions retain higher proposal quality than in a purely parallel block.

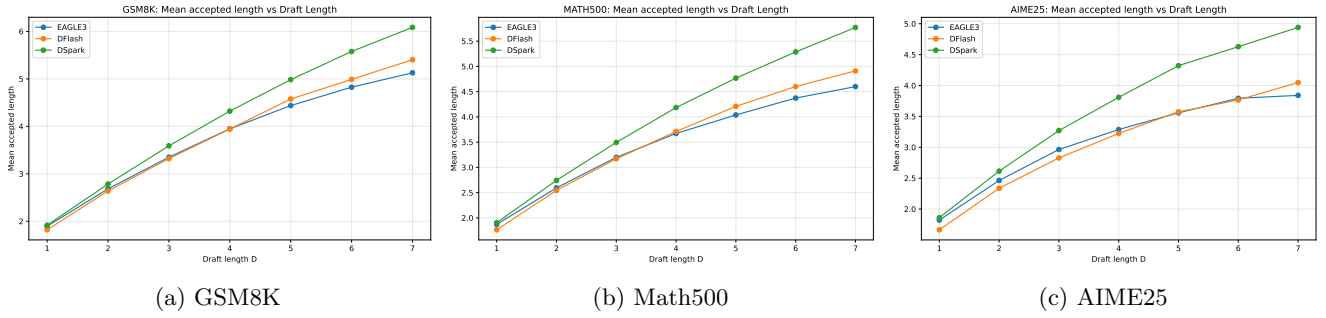


Figure 1: Mean accepted length as a function of runtime draft length D .

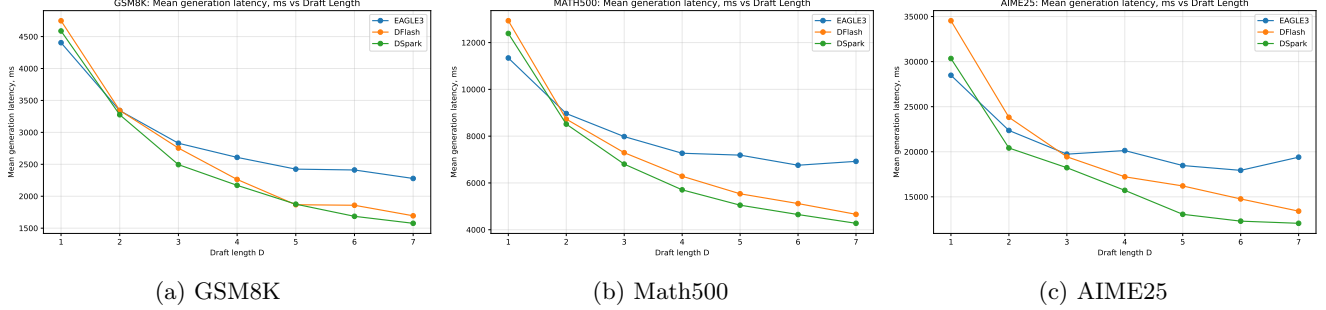


Figure 2: Mean synchronized generation latency versus runtime draft length.

At $D = 1$, the methods are much closer because there is effectively no long intra-block suffix to model. The separation appears as D increases, which is the central behavior targeted by this experiment.

Figure 2 shows the complementary latency behavior. At very short draft length EAGLE-3 is competitive and is the fastest method at $D = 1$ on the two 500-sample benchmarks. As D grows, however, its sequential draft cost limits the benefit of additional speculative tokens. DFlash scales better because a larger proposal is generated as a parallel block. DSpark combines similarly favorable long- D scaling with higher acceptance, and therefore becomes the lowest-latency method for larger D on GSM8K and Math500.

The normalized throughput curves lead to the same conclusion. On GSM8K at $D = 7$, DSpark reaches roughly 192 generated tokens/s, compared with roughly 179 for DFlash and 134 for EAGLE-3. On Math500 the corresponding values are approximately 185, 167, and 116 tokens/s. The agreement between raw latency and normalized throughput makes the ranking less sensitive to stochastic differences in output length.

AIME25 contains only 30 samples, so its curve is treated as lower-confidence supporting evidence rather than as the primary basis for fine-grained ranking.

5 Resource Efficiency and Secondary Findings

The three methods expose a clear compute-quality trade-off. EAGLE-3 uses strong sequential dependency modeling but pays sequential draft cost as D grows. DFlash minimizes this cost through parallel block generation, but has weaker explicit dependency modeling inside the block. DSpark adds a lightweight sequential dependency mechanism to a parallel backbone, increasing draft work relative to DFlash but improving the number of useful tokens obtained per target verification. On the tested workload, this additional draft-side work is repaid by higher acceptance at moderate and large D .

We also tested whether DSpark’s auxiliary confidence-metric recorder materially affected latency. A first blockwise ON→OFF experiment suggested a +7.44% difference, while reversing the order produced −0.65%. Generated outputs and acceptance statistics were identical. Because the sign changed with run order, the experiment was confounded by run-level performance drift and does not support a causal recorder-overhead estimate. We therefore retained the default DeepSpec recorder behavior in the main benchmark.

The experiment also exposed an implementation detail relevant to interpreting DeepSpec results: DFlash and DSpark share evaluator plumbing, but are not the same drafter. The DFlash checkpoint disables the Markov and confidence

mechanisms, whereas DSpark enables the semi-autoregressive dependency module and confidence head. Sharing an evaluator class therefore does not imply equivalent proposal computation.

6 Limitations and Conclusion

This study uses a single target-model size and one GPU class, and it measures offline single-request generation rather than high-concurrency production serving. The released checkpoints were trained for maximum draft length seven; our sweep is an inference-time ablation of those checkpoints rather than seven separately trained models. AIME25 is also substantially smaller than GSM8K and Math500.

Within these constraints, the main result is consistent across the two large benchmarks: EAGLE-3 is competitive for very short drafts, DFlash benefits strongly from parallel block drafting as D increases, and DSpark provides the best joint accepted-length/latency trade-off at moderate-to-large draft lengths. In particular, the advantage of DSpark grows with D , supporting the motivation for combining parallel drafting with explicit intra-block dependency modeling. For this Qwen3-4B / RTX 3090 setup, the practically strongest operating region is therefore the larger-draft regime, where DSpark converts a larger fraction of each verification into committed output while retaining favorable draft cost.

References

- [1] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- [2] Xin Cheng, Xingkai Yu, Chenze Shao, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026.
- [3] DeepSeek-AI. Deepspec: A full-stack codebase for training and evaluating speculative decoding algorithms. <https://github.com/deepseek-ai/DeepSpec>, 2026.
- [4] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.
- [5] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.